

FIXED DEPOSIT

Microservice Platform

Architecture, implementation, integration and demonstration report

Project	Submission
Module	Fixed Deposit (FD) — Group 9
Group lead	Krishna Gupta
Course context	Banking Technology Laboratory
Repository	github.com/krishna1211711/fixed-deposit-microservice
Integration baseline	github.com/kashvikumaar/BT-Team-1
Prepared	September 2026

This submission is designed for a reproducible local lab demonstration today and for extension into a portfolio-grade, independently scalable banking platform later.

Executive summary

The project implements a complete fixed-deposit lifecycle as independently deployable services behind an API gateway. A Spring Boot domain service owns account, product, interest, maturity, withdrawal and statement logic. Angular provides role-aware workflows. Separate Python reporting and notification services handle read-oriented exports and email delivery. Apache Kafka carries versioned lifecycle events, MySQL persists operational data, and Mailpit captures email without paid credentials.

Outcome	Delivered capability
Reproducible demo	One Docker Compose command starts eight containers with health checks.
Banking lifecycle	Create → transact → accrue → mature/withdraw → notify → report.
Security	JWT authentication, role restrictions, customer ownership checks, no committed live secrets.
International readiness	INR, USD, EUR, GBP, JPY, AED and KWD with ISO-aware minor units.
Scale path	Stateless gateway/services, Kubernetes manifests and HPA definitions.
Evidence	27 backend tests, frontend production build, container and API smoke tests.

Design principles

- Services own a bounded responsibility and communicate through versionable HTTP contracts.
- Money is stored using decimal arithmetic; currency validation happens at service boundaries.
- Scheduled jobs are also exposed as administrator-only demo controls for deterministic evaluation.
- Optional external dependencies have free local substitutes, so the professor needs no cloud account.

02 • SCOPE

Requirements and traceability

The lab manuals were treated as minimum requirements. The implementation adds integration, security, observability, internationalization and deployment assets above that baseline.

Requirement	Implementation evidence	Status
FD creation	Validated product, customer, branch, term, amount and opening transaction	Complete
Interest	Simple/compound calculator; scheduled daily accrual and capitalization	Complete
Maturity	Maturity processor, payout transaction, manual close and notification	Complete
Premature withdrawal	Penalty calculation, final payout, closure transactions	Complete
Statements	Idempotent daily and final-statement generation	Complete
Reports	Spring summaries plus Python CSV/PDF/PNG exports	Complete
Notifications	Audited email delivery through free local Mailpit	Complete
Team integration	Shared Swagger mapped; unresolved live payloads documented	Contract-ready

Out of scope for this lab submission

Real core-banking settlement, paid SMS/WhatsApp delivery, public-cloud production deployment and the other teams' unavailable source services are intentionally external. Extension points and configuration are included without inventing contracts that the owners have not supplied.

03 • ARCHITECTURE

Solution architecture

The application follows a microservices architecture. Each runtime can be replaced or scaled independently; only the database remains a shared local-demo dependency. Kafka already separates lifecycle processing from notification delivery. A production evolution would give each bounded context its own database and add a transactional outbox for guaranteed event publication.

Layer / service	Responsibility	Local port	Scale characteristic
Angular UI	Role-aware screens and client validation	4200	Static replicas/CDN
API Gateway	Routing and JWT pre-validation	9090	Stateless horizontal
FD Service	Products, accounts, lifecycle and schedules	8080	Stateless horizontal
Report Service	CSV, PDF and chart generation	5000	Independent workers
Kafka	Three-partition FD lifecycle event stream	9092	Partition/consumer scaling
Notification Service	Idempotent event consumption and email	5001	Consumer-group workers
MySQL	Transactional persistence + Flyway	internal	Managed DB/replica
Mailpit	Free demonstration SMTP inbox	8025	Dev-only adapter

```

Browser :4200 → Gateway :9090 → FD Service :8080 → MySQL
                        ↘ Report Service :5000 → MySQL
FD Service → Kafka :9092 → Notification Service :5001 → Mailpit :1025

```

Deployment profiles

Docker Compose is the canonical professor workflow. Kubernetes resources cover deployments, services, configuration, secret references and horizontal pod autoscaling. Cloud deployment remains provider-neutral; Azure for Students is a compatible future target but is not required to grade the project.

Domain and data model

Aggregate / table	Key fields	Purpose
users	username, password_hash, role	Authentication and authorization
customer_profile	customer_id, user_id, category	Customer identity and rate categories
products	code, currency, term/rate limits	Versionable FD product catalogue
fd_accounts	account_no, principal, rate, maturity	FD aggregate and lifecycle state
fd_transactions	type, amount, currency, GL legs	Immutable financial audit trail
fd_interest_transactions	date, interest, cumulative	Accrual audit trail
fd_statements	opening, credit, closing	Periodic customer reporting
notification_log	event, channel, status, body	Delivery evidence and retries

Integrity choices

- Flyway migrations provide deterministic schema evolution.
- BigDecimal and DECIMAL columns prevent floating-point money errors.
- Account numbers are branch-prefixed and sequence-backed.
- UUIDs provide external traceability while business account numbers remain readable.
- Transactions and notification records preserve lifecycle evidence.

Lifecycle workflows

Workflow	Validation and processing	Result
Open FD	Officer/Admin validates customer, product, term, currency, scale and minimum	ACTIVE account + DEPOSIT transaction + email
Daily accrual	Scheduler loads ACTIVE accounts and calculates per-day interest	Accrual row + account accrued balance
Capitalization	Frequency boundary reached	Interest becomes principal + transaction
Maturity	Business date reaches maturity date	Payout transaction + CLOSED state + Kafka email event
Premature close	Customer ownership/role, ACTIVE state and penalty rules	Withdrawal + penalty audit + CLOSED state
Statement	Daily/admin trigger computes opening, credit and closing	Idempotent statement retrievable by account

Deterministic lab testing

The time-travel endpoint advances the logical business date and invokes lifecycle jobs without changing the computer clock. This makes interest, maturity and statement scenarios repeatable during a short demonstration.

API surface

Capability	Representative endpoint	Access
Authentication	POST /api/auth/login	Public
Products	GET /api/product/search	Authenticated
Calculator	POST /api/fd/calculate	Customer
Create FD	POST /api/fd/account/create-with-txn	Officer/Admin
My accounts	GET /api/fd/accounts/my	Customer
Transactions	GET /api/fd/account/{id}/transactions	Owner/Staff
Withdraw	POST /api/fd/account/withdraw	Owner/Officer
Batches	POST /api/admin/batch/*	Admin
Exports	GET /reports/fd-summary.{csv pdf}	JWT via gateway

An OpenAPI file is included for FD-specific consumers. The shared Team-1 Swagger is mapped in docs/INTEGRATION.md, including differences that require confirmation from the owning groups before a live merge.

Security and privacy

Control	Implementation
Authentication	BCrypt password hashes and signed JWTs with role/customer claims
Authorization	ADMIN, BANK_OFFICER and CUSTOMER method rules
Object ownership	Customer account/detail/transaction/statement/withdrawal access checks
Secrets	Environment variables; example Kubernetes Secret; live secret file ignored
PII	Masked Aadhaar field and minimal customer exposure
Audit	Financial transactions, interest rows and notification outcomes
Edge protection	Gateway JWT validation, rate-limiter component and health endpoints

Security action required

The repository's original public commit contained an SMTP credential-like value. Removing it from the latest tree does not remove it from Git history. If genuine, revoke it immediately. A history rewrite can be performed separately only after coordinating with collaborators because it changes commit identities.

Multi-currency correctness

CurrencyRules centralizes supported ISO 4217-like precision. Validation rejects unsupported currencies, product/account currency mismatches and principal amounts with excessive fractional digits.

Currency	Typical use	Minor units	Example valid amount
INR	Indian rupee	2	100000.00
USD	US dollar	2	2500.50
EUR	Euro	2	2500.50
GBP	Pound sterling	2	2500.50
JPY	Japanese yen	0	250000
AED	UAE dirham	2	10000.00
KWD	Kuwaiti dinar	3	1500.125

Database money columns use scale 3 so KWD values are retained. Service calculations round to the account currency's permitted scale before persistence and response serialization.

Reporting and notifications

Artifact / event	Provider	Demonstration behavior
FD summary CSV	Python report service	Downloads a machine-readable portfolio snapshot
FD summary PDF	Python report service	Downloads a printable management summary
Portfolio PNG	Python report service	Renders customer allocation visualization
Spring report API	FD service	Role-specific JSON summaries and CSV
FD opened	Kafka + notification service	Versioned event + SMTP + idempotent audit record
Interest/maturity/withdrawal	Kafka + notification service	Event-specific customer message and status

Mailpit is deliberately used for grading: it is free, local, contains no real recipient data and gives visible proof of delivery at <http://localhost:8025>. Production email, SMS or WhatsApp can be introduced behind the same notification boundary later.

Verification evidence

Check	Result	Evidence
Spring Boot tests	27 passed	Unit, scheduler and integration suites
Gateway build	Passed	Maven package/test lifecycle
Angular build	Passed	Angular 21 production compilation
Production npm audit	0 findings	npm audit --omit=dev --audit-level=moderate
Python validation	Passed	py_compile for reporting and notifications
Docker images	Built	FD, gateway, UI, reporting and notification images
Container health	8 running	Application services, MySQL, Kafka and Mailpit
End-to-end smoke	Passed	Login, products, create, transactions, batch, email, reports

Known development-only audit note

The full npm audit reports seven moderate findings through development-server tooling. The production dependency audit is clean. These packages are not shipped in the Nginx runtime image; upgrades should still be monitored in future maintenance.

11 • OPERATIONS

Containers and Kubernetes

```
git clone https://github.com/krishna1211711/fixed-deposit-microservice.git
cd fixed-deposit-microservice
copy .env.example .env          # optional: replace demo values
docker compose up --build -d
docker compose ps
```

URL	Purpose
http://localhost:4200	Angular demonstration UI
http://localhost:9090	API gateway
http://localhost:8080/swagger-ui.html	FD API documentation
http://localhost:8025	Mailpit demonstration inbox
http://localhost:9090/reports/health	Reporting health through gateway

Kubernetes manifests include service/deployment definitions, ConfigMaps, Secret references and HPAs for the gateway, FD service and report service. Images and a managed database must be supplied by the selected cloud environment.

Cross-team integration

The Team-1 repository currently provides an ER diagram, a shared Swagger contract and a work-distribution workbook, but no runnable Auth, Customer, Calculation, Product or Transaction services. The FD module therefore preserves a stable adapter boundary and documents the exact data still required.

Dependency	Current strategy	Required from owning group
Auth	Local JWT issuer; gateway validates same secret	Issuer URL, claims, keys/JWKS and roles
Customer	Local customer_profile for deterministic demo	Lookup endpoint, success/error payloads
Calculation	Built-in compound/simple engine	calcld schema and rate response contract
Product	Local product catalogue	Product identifiers, currencies and term/rate fields
Transactions/GL	Local immutable ledger entries	Posting endpoint, idempotency and GL account rules

The shared POST /api/v1/accounts request contains accountName and calcld, which is not sufficient to open an FD without customer, product, amount, term, currency and branch data. This mismatch is explicitly recorded rather than silently guessing financial values.

13 • EVALUATION

Demonstration plan

Minute	Action	Evidence shown
0–1	Start Compose and show healthy services	docker compose ps
1–2	Login as customer/officer/admin	Role-specific navigation
2–3	Simulate compound interest	Maturity and interest result
3–5	Create an FD as officer	Account number + opening transaction
5–6	View it as owning customer	Ownership and transaction history
6–7	Run interest/time travel as admin	Accrual/lifecycle changes
7–8	Open Mailpit and download reports	Email + CSV/PDF evidence
8–9	Show Kubernetes/CI/integration docs	Scale and team-readiness

Demo credentials are deliberately local and documented in the README. They must never be reused in a hosted deployment. A separate demonstration script accompanies the submission.

Team contribution map

Member	Assigned focus from workbook
Vritika Vinod	Schema, account generation, account creation and UI
Anirudh Syam Menon	Transaction schema, initial deposit, GL and UI
Parth Gupta	Interest engine, scheduler, statements and dashboard
Niranjan Subhash Nair	Premature withdrawal
Pratham P Shetty	Maturity, payout and manual close
Krishna Gupta	Email/WhatsApp notification scope, reporting API, time travel and lifecycle tests; group lead and integration

This table mirrors the detailed Group-9 allocation in Work_Distribution_TEAM_1.xlsx. Git history and pull requests should be used as the final authorship evidence when individual branches are integrated.

Limitations and roadmap

Horizon	Recommended enhancement
Before final team merge	Confirm external payloads; add consumer-driven contract tests; resolve shared account endpoint semantics.
Portfolio polish	Add Testcontainers, OpenTelemetry traces, Grafana dashboards and architecture decision records.
Production security	OIDC provider (Microsoft Entra ID/Keycloak), asymmetric JWTs, vault-managed secrets and TLS.
Scale	Database-per-service, transactional outbox, Kafka retry/DLQ topics and report read replicas.
Cloud	Use Azure for Students if available; otherwise retain provider-neutral Kubernetes or a free container host.
Notifications	Replace Mailpit with verified email; add WhatsApp/SMS only within free student quotas.

Submission readiness

The local demonstration is complete and repeatable. Remaining work depends on people or systems outside this repository: professor-specific formatting, final team contracts, cloud entitlement confirmation and revocation of any credential previously exposed in public history.